

B12705017_曾文儀_part1

資管三 B12705017 曾文儀

環境說明

作業系統：macOS (Darwin 24.5.0) — 開發與測試環境

- Linux (Ubuntu 等) 相容；支援其他 POSIX 的類 Unix 系統

編譯器：g++ (GNU C++ Compiler)

- 需支援 C++11 (使用 `std=c++11`)、pthread (使用 `pthread`)

函式庫

- 標準 C++ 函式庫：`iostream`, `string`, `vector`, `map`, `sstream` 等
- POSIX Socket API：`sys/socket.h`, `netinet/in.h`, `arpa/inet.h`
- POSIX Thread：pthread (透過 `pthread` 連結)
- Unix 系統呼叫：`unistd.h` (`close`, `read`, `write` 等)

系統需求

- 不需要額外安裝第三方套件
- 使用系統內建的標準函式庫與 API
- 需要網路連線 (TCP/IP)

編譯選項 (Makefile 中已設定)

- `std=c++11`：使用 C++11 標準
- `Wall -Wextra`：啟用警告
- `pthread`：連結 pthread 函式庫
- `O2`：最佳化等級 2

測試環境

- 至少需要三個終端機視窗 (一個執行 Server，兩個執行 Client)
- 確保使用的 port 沒有被其他程式佔用

編譯與執行

1. 開啟終端機，切換到專案目錄後，執行 `make` 編譯

- 會自動使用 Makefile 編譯 `client.cpp` → 成功後產生 `client` 可執行檔
- 如果要清除編譯產物：`make clean`，刪除 `client` 和 `client.o` (或是 `make rebuild`)

2. 編譯完成後，以 `./client` 執行程式

3. 程式啟動後，會要求輸入連線資訊：

- Server IP address：(本機測試用 `127.0.0.1`)
- Server Port：Server 監聽的 port (例如 `8888`)
- Your listening port for P2P connections：自己 (client) 的 P2P 監聽 port (與 Server 不同)

4. 使用主選單，輸入數字執行不同功能

- `1` - Register (註冊)：輸入使用者名稱與初始餘額；註冊使用持久連線。
- `2` - Login (登入)：程式會接收伺服器回傳的帳戶餘額、伺服器公鑰、以及目前在線的使用者；登入後才能使用其他功能。
- `3` - List (查詢餘額和線上使用者)：可以隨時呼叫，用於確認餘額是否正確更新。

- **4** - Transfer (轉帳給其他使用者)：向其他在線使用者發送轉帳請求。
 - 付款方直接連線到收款方的 Client (不經過 Server)、立即扣除本地餘額
 - 收款方收到轉帳訊息後立即增加本地餘額、自動向 Server 報告交易，更新雙方帳戶餘額
- **5** - Exit (離線並結束程式)：程式會通知伺服器並關閉所有連線。

5. 結束程式

- 選擇選單 **5** 離線 (不要直接按 Ctrl+C 強制結束)

程式的架構與流程

1. 在程式啟動後，會立即建立與伺服器的持久連線 (persistent connection)。這個連線會用於所有後續操作 (註冊、登入、查詢、離線)，直到程式結束。同時，程式會啟動一個背景執行緒來監聽其他用戶的 P2P 轉帳請求。
2. 多執行緒架構：
 - **主執行緒 (Main Thread)**：負責使用者介面的呈現和互動，包括顯示選單、接收使用者輸入、執行對應的功能 (註冊、登入、查詢、轉帳、離線)、以及與 Server 的通訊。
 - **監聽執行緒 (Listener Thread)**：在程式啟動時建立並在背景持續執行，負責監聽指定的 port，等待其他 Client 的連線請求。當有其他 Client 要轉帳時，他們會連接到這個 port。
 - **處理執行緒 (Handler Threads)**：當監聽執行緒接受一個新的轉帳連線時，會動態建立一個新的處理執行緒來處理該筆轉帳。這個執行緒會接收轉帳訊息、解析內容、更新本地餘額、向 Server 報告交易、然後結束。
 - **同步機制**：
 - `cout_mutex`：保護標準輸出，避免多個執行緒同時輸出造成混亂
 - `socket_mutex`：保護 `server_socket`，避免多個執行緒同時使用造成資料混亂
 - `users_mutex`：保護 `online_users` map，確保執行緒安全
3. 網路通訊
 - **與伺服器的通訊**：使用 TCP Socket 進行與伺服器之間的通訊。程式在啟動時建立一個持久連線，所有操作 (註冊、登入、查詢、離線) 都使用這個連線。
 - **P2P 通訊**：轉帳時，付款方會建立一個新的 TCP Socket 連線直接連接到收款方的 Client，發送轉帳訊息後即關閉連線。這是典型的 P2P 通訊架構。
 - **訊息格式**：所有訊息使用 ASCII 文字編碼，使用 `#` 符號分隔欄位，並以 `\r\n` (CRLF) 作為訊息結束標記 (部分命令如 `List` 和 `Exit` 不使用 CRLF)。

程式碼說明

標頭檔與常數定義

```
#include <iostream>           // 標準 C++ 函式庫，用於輸入輸出、字串處理、容器操作等
#include <string>
#include <cstring>
#include <algorithm>
#include <vector>
#include <map>
#include <thread>              // C++11 標準執行緒函式庫，用於多執行緒程式設計和同步
#include <mutex>
#include <chrono>
#include <sstream>             // POSIX Socket API，用於網路通訊
#include <sys/socket.h>
#include <netinet/in.h>
```

```

#include <arpa/inet.h>
#include <unistd.h>          // Unix 系統呼叫和錯誤處理
#include <errno.h>

using namespace std;

// Constants
#define BUFFER_SIZE 4096    // Maximum size for network messages
#define CRLF "\r\n"         // Carriage Return + Line Feed (protocol requirement)

```

全域變數與資料結構

```

// Global variables for network connections
int server_socket = -1;    // Socket for persistent connection to server
int listen_socket = -1;    // Socket for listening to P2P connections from other clients
string username = "";      // Current logged-in username
string server_ip = "";     // Server's IP address
int server_port = 0;       // Server's port number
int my_port = 0;           // Our listening port for P2P connections

// Global variables for account state
int account_balance = 10000; // Current account balance (initialized with default)
string server_public_key = ""; // Server's public key (for Phase 2)
bool is_logged_in = false;    // Login status flag
bool is_running = true;       // Main loop control flag

// Mutex locks for thread synchronization
mutex cout_mutex;    // Protects console output from multiple threads
mutex socket_mutex;  // Protects server_socket from concurrent access
mutex users_mutex;   // Protects online_users map from concurrent access

// Structure to hold online user information
struct OnlineUser {
    string username; // User's account name
    string ip;       // User's IP address
    int port;        // User's listening port for P2P connections
};

// Map to store all currently online users (username -> OnlineUser)
map<string, OnlineUser> online_users;

```

函式宣告與實作

1. 註冊功能 (handle_register)

```

void handle_register() {
    // Check if server connection is available
    if (server_socket == -1) {
        cout << "Error: Not connected to server." << endl;
        return;
    }

    // Check if already logged in

```

```

if (is_logged_in) {
    cout << "You are already logged in. Please logout first..." << endl;
    return;
}

cout << "Enter username: ";
string user;
getline(cin, user);

cout << "Enter initial deposit amount: ";
string amount_str;
getline(cin, amount_str);
int amount = stoi(amount_str);

// Send registration message: REGISTER#username#amount\r\n
string message = "REGISTER#" + user + "#" + to_string(amount) + CRLF;

if (!send_message(server_socket, message)) {
    cout << "Failed to send registration request." << endl;
    return;
}

// Receive response from server
string response = receive_message(server_socket);

if (response.empty()) {
    cout << "No response from server." << endl;
    return;
}

// Parse response
if (response.find("100 OK") != string::npos) {
    cout << "\nRegistration successful!" << endl;
} else if (response.find("210 FAIL") != string::npos) {
    cout << "\nRegistration failed. Username may already exist." << endl;
} else {
    cout << "\nUnexpected response: " << response << endl;
}
}

```

- 功能：向伺服器發送註冊請求，並根據伺服器回應判斷註冊是否成功。
- 流程：
 1. 構建註冊訊息 "REGISTER#username#amount\r\n" 並使用持久連線發送給伺服器
 2. 接收伺服器回應並解析：
 - "100 OK" 表示註冊成功
 - "210 FAIL" 表示註冊失敗，可能是帳號已存在

2. 登入功能 (handle_login)

```

void handle_login() {
    // Check if server connection is available
    if (server_socket == -1) {

```

```

        cout << "Error: Not connected to server." << endl;
        return;
    }

    // Check if already logged in
    if (is_logged_in) {
        cout << "You are already logged in..." << endl;
        return;
    }

    cout << "Enter username: ";
    getline(cin, username);

    // Send login message: username#port\r\n
    string message = username + "#" + to_string(my_port) + CRLF;
    if (!send_message(server_socket, message)) {
        cout << "Failed to send login request." << endl;
        username = "";
        return;
    }

    // Receive response from server
    string response = receive_message(server_socket);

    if (response.empty()) {
        cout << "No response from server." << endl;
        username = "";
        return;
    }

    // Check for authentication failure
    if (response.find("220 AUTH_FAIL") != string::npos) {
        cout << "\nLogin failed. Please register first." << endl;
        username = "";
        return;
    }

    // Parse response to extract balance and online users
    parse_online_list(response);
    is_logged_in = true;
    cout << "\nLogin successful!" << endl;
}

```

- 功能：向伺服器發送登入請求，並根據回應更新客戶端狀態。
- 流程：
 1. 要求使用者輸入使用者名稱
 2. 構建登入訊息 `"username#port\r\n"` 並使用持久連線發送給伺服器
 3. 接收伺服器回應並解析：
 - `"220 AUTH_FAIL"` 表示登入失敗，可能是帳號未註冊

3. 解析伺服器回應的清單 (`parse_online_list`)

```

void parse_online_list(const string& response) {
    istringstream iss(response);
    string line;

    // First line: account balance
    if (getline(iss, line)) {
        // Remove any CR/LF characters
        line.erase(std::remove(line.begin(), line.end(), '\r'), line.end());
        line.erase(std::remove(line.begin(), line.end(), '\n'), line.end());

        // Check if line contains a valid number
        try {
            if (!line.empty() && (isdigit(line[0]) || line[0] == '-')) {
                account_balance = stoi(line);
                cout << "Account Balance: $" << account_balance << endl;
            } else {
                cout << "[ERROR] Invalid balance format: '" << line << "'" << endl;
                return;
            }
        } catch (const std::exception& e) {
            cout << "[ERROR] Failed to parse balance: " << e.what() << endl;
            return;
        }
    }

    // Second line: server public key
    if (getline(iss, line)) {
        line.erase(std::remove(line.begin(), line.end(), '\r'), line.end());
        line.erase(std::remove(line.begin(), line.end(), '\n'), line.end());
        server_public_key = line;
        if (!server_public_key.empty()) {
            cout << "Server Public Key: [Received]" << endl;
        }
    }

    // Third line: number of online users
    int num_users = 0;
    if (getline(iss, line)) {
        line.erase(std::remove(line.begin(), line.end(), '\r'), line.end());
        line.erase(std::remove(line.begin(), line.end(), '\n'), line.end());
        num_users = stoi(line);
        cout << "Number of online users: " << num_users << endl;
    }

    // Clear current online users map before updating
    users_mutex.lock();
    online_users.clear();

    // Parse each online user's information
    cout << "\nOnline Users:" << endl;
    cout << "-----" << endl;
    for (int i = 0; i < num_users; i++) {

```

```

        if (getline(iss, line)) {
            line.erase(std::remove(line.begin(), line.end(), '\r'), line.end());
            line.erase(std::remove(line.begin(), line.end(), '\n'), line.end());

            // Parse: username#ip#port
            size_t pos1 = line.find('#');
            size_t pos2 = line.find('#', pos1 + 1);

            if (pos1 != string::npos && pos2 != string::npos) {
                OnlineUser user;
                user.username = line.substr(0, pos1);
                user.ip = line.substr(pos1 + 1, pos2 - pos1 - 1);
                user.port = stoi(line.substr(pos2 + 1));

                // Add user to online users map
                online_users[user.username] = user;
                cout << user.username << " @ " << user.ip << ":" << user.port << endl;
            }
        }
    }
    cout << "-----" << endl;
    users_mutex.unlock();
}

```

- 功能：解析伺服器回應的帳戶餘額、伺服器公鑰、在線人數以及在線使用者列表。
- 流程：
 1. 使用 `istringstream` 按行讀取回應
 2. 依序解析：
 - 第一行為帳戶餘額，轉換為 `int` 並更新 `account_balance`
 - 第二行為伺服器公鑰，儲存到 `server_public_key`
 - 第三行為在線人數，轉換為 `int`
 - 接下來的若干行為每個在線使用者的資訊，格式為 `"username#ip#port"`，解析後存入 `online_users` map
 3. 使用 `users_mutex` 保護 `online_users` map 的更新操作
- 錯誤處理：包含完整的錯誤處理機制，包括格式檢查和例外處理。

4. 取得帳戶餘額與上線清單 (handle_list)

```

void handle_list() {
    // Check if user is logged in
    if (!is_logged_in) {
        cout << "Please login first." << endl;
        return;
    }

    cout << "\n--- Requesting updated list ---" << endl;

    // Send list request to server
    // NOTE: List command does NOT include CRLF
    string message = "List";
    if (!send_message(server_socket, message)) {

```

```

        cout << "Failed to send list request." << endl;
        return;
    }

    // Receive response from server
    string response = receive_message(server_socket);

    if (response.empty()) {
        cout << "No response from server." << endl;
        return;
    }

    // Parse and display updated information
    parse_online_list(response);
}

```

- 功能：向伺服器請求帳戶餘額與在線使用者清單，並解析伺服器回應。
- 流程：
 1. 發送訊息 `"List"` 給伺服器（不使用 CRLF）
 2. 接收並解析伺服器回應，使用 `parse_online_list` 函式解析並顯示
- 使用者完成轉帳後想確認餘額是否正確更新、或者想要查看是否有新的使用者上線時使用。

5. 轉帳功能 (handle_transfer)

```

void handle_transfer() {
    // Check if user is logged in
    if (!is_logged_in) {
        cout << "Please login first." << endl;
        return;
    }

    cout << "\n--- Transfer Money ---" << endl;

    // Show list of online users (excluding ourselves)
    users_mutex.lock();
    if (online_users.empty()) {
        cout << "No other users online." << endl;
        users_mutex.unlock();
        return;
    }

    cout << "Online users:" << endl;
    int index = 1;
    vector<string> user_list;
    for (const auto& pair : online_users) {
        // Don't show ourselves in the list
        if (pair.first != username) {
            cout << index++ << ". " << pair.first << endl;
            user_list.push_back(pair.first);
        }
    }
    users_mutex.unlock();
}

```



```

if (user_list.empty()) {
    cout << "No other users online." << endl;
    return;
}

// Get recipient username from user
cout << "Enter recipient username: ";
string recipient;
getline(cin, recipient);

// Check if recipient exists and is online
users_mutex.lock();
auto it = online_users.find(recipient);
if (it == online_users.end()) {
    cout << "User not found or not online." << endl;
    users_mutex.unlock();
    return;
}
OnlineUser target_user = it->second; // Copy recipient's connection info
users_mutex.unlock();

// Get transfer amount from user
cout << "Enter amount to transfer: ";
string amount_str;
getline(cin, amount_str);
int amount = stoi(amount_str);

// Validate amount
if (amount <= 0) {
    cout << "Invalid amount." << endl;
    return;
}

// Check if we have sufficient balance
if (amount > account_balance) {
    cout << "Insufficient balance." << endl;
    return;
}

// P2P Connection: Connect directly to recipient's client
cout << "Connecting to " << recipient << " at " << target_user.ip << ":" << target_u
ser.port << "..." << endl;
int peer_sock = connect_to_server(target_user.ip, target_user.port);
if (peer_sock == -1) {
    cout << "Failed to connect to recipient." << endl;
    return;
}

// Send transfer message directly to recipient: sender#amount#recipient
string transfer_msg = username + "#" + to_string(amount) + "#" + recipient;

if (!send_message(peer_sock, transfer_msg)) {

```

```

        cout << "Failed to send transfer request." << endl;
        close(peer_sock);
        return;
    }

    cout << "Transfer request sent to " << recipient << endl;
    close(peer_sock); // Close P2P connection after sending

    // Update local balance (deduct the transferred amount)
    account_balance -= amount;
    cout << "\nTransfer completed. Your new balance: $" << account_balance << endl;

    // Wait for recipient to process
    this_thread::sleep_for(chrono::milliseconds(500));

    // Optionally request updated info from server
    cout << "\nRequesting updated list from server..." << endl;
    string list_msg = "List";
    if (send_message(server_socket, list_msg)) {
        string response = receive_message(server_socket);
        if (!response.empty()) {
            parse_online_list(response);
            cout << "Balance updated after transfer." << endl;
        } else {
            cout << "Warning: Could not retrieve updated balance from server." << endl;
        }
    } else {
        cout << "Warning: Failed to send list request to server." << endl;
    }
}

```

- 功能：實現使用者之間的 P2P 轉帳操作。
- 流程：
 1. 顯示目前在線的其他使用者清單（不包含自己），請使用者輸入收款人和轉帳金額
 2. 檢查收款人是否在線上，以及轉帳金額是否有效且不超過餘額
 3. 建立與收款人的 P2P 連線，發送轉帳訊息 `"sender#amount#recipient"`
 4. 立即扣除本地餘額並顯示更新後的餘額
 5. 等待收款方處理後，可選地向伺服器查詢更新後的清單以確認餘額同步

6. 離線通知 (handle_exit)

```

void handle_exit() {
    cout << "\n--- Exiting ---" << endl;

    // If logged in, send proper logout notification to server
    if (is_logged_in && server_socket != -1) {
        cout << "Logging out..." << endl;

        // Send exit message to server
        // NOTE: Exit command does NOT include CRLF
        string message = "Exit";
    }
}

```

```

        if (send_message(server_socket, message)) {
            // Wait for server's goodbye response
            string response = receive_message(server_socket);
            if (response.find("Bye") != string::npos) {
                cout << "Logged out successfully." << endl;
            }
        }
    }

    // Close persistent server connection
    if (server_socket != -1) {
        close(server_socket);
        server_socket = -1;
    }

    // Mark logged out and stop the program
    is_logged_in = false;
    is_running = false;
    cout << "Goodbye!" << endl;
}

```

- 功能：通知伺服器使用者即將離線。
- 流程：
 1. 如果已登入，發送 `"Exit"` 訊息給伺服器（注意：不使用 CRLF）
 2. 等待伺服器回應 `"Bye"` 確認
 3. 關閉所有連線（`server_socket` 和 `listen_socket`）
 4. 更新狀態旗標並結束程式

7. 網路通訊輔助函式

```

int connect_to_server(const string& ip, int port) {
    // Create TCP socket
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == -1) {
        perror("socket");
        return -1;
    }

    // Set up server address structure
    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port); // Convert port to network byte order

    // Convert IP address from string to binary form
    if (inet_pton(AF_INET, ip.c_str(), &server_addr.sin_addr) <= 0) {
        perror("inet_pton");
        close(sock);
        return -1;
    }

    // Connect to server

```

```

    if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
        perror("connect");
        close(sock);
        return -1;
    }

    return sock;
}

```

- 功能：建立一個 TCP Socket 並連接到指定的 IP 和 port。

- 流程：

1. 建立 TCP socket
2. 設定伺服器位址結構（使用 `sockaddr_in`）
3. 將 port 轉換為網路位元組順序（使用 `htons`）
4. 將 IP 位址從字串轉換為二進位形式（使用 `inet_pton`）
5. 連接到伺服器（使用 `connect`）

- 成功時回傳 socket 描述符，失敗時回傳 `-1`。

```

bool send_message(int sock, const string& message) {
    ssize_t sent = send(sock, message.c_str(), message.length(), 0);
    if (sent == -1) {
        perror("send");
        return false;
    }
    return true;
}

```

- 功能：透過指定的 socket 發送訊息。

- 流程：

1. 使用 `send` 函式發送訊息
2. 檢查是否發送成功

- 回傳值：成功時回傳 `true`，失敗時回傳 `false`。

```

string receive_message(int sock) {
    char buffer[BUFFER_SIZE];
    memset(buffer, 0, BUFFER_SIZE);

    ssize_t received = recv(sock, buffer, BUFFER_SIZE - 1, 0);

    if (received == -1) {
        perror("recv");
        return "";
    } else if (received == 0) {
        // Connection closed by peer
        return "";
    }
}

```

```
    return string(buffer, received);  
}
```

- 功能：從指定的 socket 接收訊息。
- 流程：
 1. 建立緩衝區並初始化為零
 2. 使用 `recv` 函式接收資料
 3. 檢查接收結果：`-1` 表示錯誤、`0` 表示連線已關閉、其他值表示成功接收
- 成功時回傳接收到的訊息字串，失敗時回傳空字串。

8. 監聽執行緒 (listener_thread)

```
void listener_thread() {  
    // Create listening socket  
    listen_socket = socket(AF_INET, SOCK_STREAM, 0);  
    if (listen_socket == -1) {  
        perror("listener socket");  
        return;  
    }  
  
    // Set socket option to reuse address (useful for quick restart)  
    int opt = 1;  
    if (setsockopt(listen_socket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) == -1) {  
        perror("setsockopt");  
        close(listen_socket);  
        return;  
    }  
  
    // Bind socket to our listening port  
    struct sockaddr_in listen_addr;  
    memset(&listen_addr, 0, sizeof(listen_addr));  
    listen_addr.sin_family = AF_INET;  
    listen_addr.sin_addr.s_addr = INADDR_ANY; // Listen on all network interfaces  
    listen_addr.sin_port = htons(my_port);    // Convert port to network byte order  
  
    if (::bind(listen_socket, (struct sockaddr*)&listen_addr, sizeof(listen_addr)) == -  
1) {  
        perror("bind");  
        close(listen_socket);  
        return;  
    }  
  
    // Start listening for connections (queue up to 5 pending connections)  
    if (listen(listen_socket, 5) == -1) {  
        perror("listen");  
        close(listen_socket);  
        return;  
    }  
  
    safe_print("P2P listener started on port " + to_string(my_port));  
  
    // Accept incoming connections in a loop
```

```

while (is_running) {
    struct sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);

    // Accept incoming connection (blocking call)
    int client_sock = accept(listen_socket, (struct sockaddr*)&client_addr, &client_
len);
    if (client_sock == -1) {
        if (is_running) {
            perror("accept");
        }
        continue;
    }

    // Handle this client connection in a new thread
    thread handler(handle_client_connection, client_sock);
    handler.detach(); // Detach so handler runs independently
}
}

```

- 功能：在背景持續執行的監聽執行緒，用於接收其他使用者的 P2P 轉帳請求。
- 流程：

1. 建立監聽 Socket
2. 設定 Socket 選項 `SO_REUSEADDR`，以便快速重啟 Socket
3. 綁定 Socket 到使用者指定的 port (`my_port`)
4. 開始監聽連線（最多佇列 5 個待處理連線）
5. 在迴圈中接受連線請求（使用 `accept`，這是阻塞呼叫）
6. 對每個接受的連線，建立一個新的處理執行緒來處理轉帳訊息

9. 處理客戶端連線 (`handle_client_connection`)

```

void handle_client_connection(int client_sock) {
    // Receive transfer message from peer
    string message = receive_message(client_sock);

    if (message.empty()) {
        safe_print("[ERROR] Empty P2P message received");
        close(client_sock);
        return;
    }

    // Parse transfer message: sender#amount#recipient
    size_t pos1 = message.find('#');
    size_t pos2 = message.find('#', pos1 + 1);

    if (pos1 != string::npos && pos2 != string::npos) {
        string sender = message.substr(0, pos1);
        string amount_str = message.substr(pos1 + 1, pos2 - pos1 - 1);
        string recipient = message.substr(pos2 + 1);

        // Remove CRLF from recipient field

```

```

        recipient.erase(std::remove(recipient.begin(), recipient.end(), '\r'), recipient.
t.end());
        recipient.erase(std::remove(recipient.begin(), recipient.end(), '\n'), recipient.
t.end());

        int amount = stoi(amount_str);

        // Display transfer notification to user
        safe_print("\n*** Incoming Transfer ***");
        safe_print("From: " + sender);
        safe_print("Amount: $" + to_string(amount));
        safe_print("To: " + recipient);
        safe_print("*****\n");

        // Update local balance (optimistic update)
        account_balance += amount;
        safe_print("Transfer received. New balance: $" + to_string(account_balance));

        // Report transaction to server so it can update both accounts
        if (is_logged_in && server_socket != -1) {
            socket_mutex.lock(); // Lock because main thread may also use server_socket

            // Format: sender#amount#recipient\r\n
            string transaction_msg = sender + "#" + amount_str + "#" + recipient + CRLF;

            if (send_message(server_socket, transaction_msg)) {
                string response = receive_message(server_socket);
                if (!response.empty()) {
                    safe_print("[DEBUG] Server response: '" + response + "'");
                }
                safe_print("Transaction reported to server.");
            } else {
                safe_print("[WARNING] Failed to report transaction to server");
            }

            socket_mutex.unlock();
        } else {
            safe_print("Warning: Not logged in, transaction not reported to server");
        }
    } else {
        safe_print("[ERROR] Failed to parse P2P transfer message. Invalid format.");
    }

    // Close P2P connection
    close(client_sock);
}

```

- 功能：處理來自其他客戶端的 P2P 轉帳請求。

- 流程：

1. 接收轉帳訊息，解析訊息格式 `"sender#amount#recipient"`，移除可能的 CRLF 字元
2. 顯示轉帳通知給使用者、立即增加本地餘額（樂觀更新）

3. 向伺服器報告交易（格式：`sender#amount#recipient\r\n`），讓伺服器更新雙方帳戶餘額

4. 關閉 P2P 連

10. 執行緒安全輸出 (`safe_print`)

```
void safe_print(const string& message) {  
    lock_guard<mutex> lock(cout_mutex);  
    cout << message << endl;  
}
```

- 功能：執行緒安全的輸出函式，使用 mutex 防止輸出混亂。
- 流程：
 1. 使用 `lock_guard` 自動鎖定 `cout_mutex`
 2. 輸出訊息
 3. `lock_guard` 在函式結束時自動解鎖
- 所有執行緒（主執行緒、監聽執行緒、處理執行緒）都使用此函式來輸出訊息。

main function

```
int main() {  
    cout << "=====" << endl;  
    cout << "    P2P Micropayment System - Client    " << endl;  
    cout << "=====" << endl;  
    cout << endl;  
  
    // Get server connection information from user  
    cout << "Enter Server IP address: ";  
    getline(cin, server_ip);  
  
    cout << "Enter Server Port: ";  
    string port_str;  
    getline(cin, port_str);  
    server_port = stoi(port_str);  
  
    // Get our listening port for accepting P2P connections  
    cout << "Enter your listening port for P2P connections: ";  
    getline(cin, port_str);  
    my_port = stoi(port_str);  
  
    // Establish persistent connection to server at startup  
    cout << "\nConnecting to server..." << endl;  
    server_socket = connect_to_server(server_ip, server_port);  
    if (server_socket == -1) {  
        cout << "Failed to connect to server. Exiting." << endl;  
        return 1;  
    }  
    cout << "Connected to server successfully!" << endl;  
    cout << "You can now Register (if new user) or Login (if existing user)." << endl;  
  
    // Start listener thread for P2P connections in background  
    thread listener(listener_thread);
```



```

listener.detach(); // Detach so it runs independently

// Give listener thread time to initialize and start listening
this_thread::sleep_for(chrono::milliseconds(500));

// Main menu loop - continues until user chooses to exit
while (is_running) {
    print_menu();

    string choice;
    cout << "\nEnter your choice: ";
    getline(cin, choice);

    // Dispatch to appropriate handler based on user choice
    if (choice == "1") {
        handle_register();
    } else if (choice == "2") {
        handle_login();
    } else if (choice == "3") {
        handle_list();
    } else if (choice == "4") {
        handle_transfer();
    } else if (choice == "5") {
        handle_exit();
    } else {
        cout << "Invalid choice. Please try again." << endl;
    }
}

// Cleanup: close all sockets before exiting
if (server_socket != -1) {
    close(server_socket);
}
if (listen_socket != -1) {
    close(listen_socket);
}

return 0;
}

```

- 功能：程式的入口點，提供使用者介面並根據使用者選擇執行相應功能。
- 流程：
 1. 初始化：
 - 顯示程式標題
 - 要求使用者輸入伺服器的 IP 位址和 port number、自己的 P2P 監聽 port
 - 建立與伺服器的持久連線啟動監聽執行緒來接收其他客戶端的轉帳請求
 2. 主迴圈：
 - 顯示選單並接收使用者選擇
 - 根據選擇執行操作

3. 清理：在退出前關閉所有 Socket 連線（`server_socket` 和 `listen_socket`）

- 使用持久連線模式，所有操作都使用同一個 `server_socket`
- 使用多執行緒架構，監聽執行緒在背景持續執行
- 使用 `is_running` 旗標控制程式執行，確保優雅地結束